

Scaling Agents for Computer Use

Wide Scaling & Behavior Judge

발제자 : 허예린

날짜 : 2026/09/11

목차 페이지

- 01 Agent가 뭐야?
- 02 용어 정리
- 03 기존 Agent의 문제 + POMDP
- 04 Scaling이 뭐야?
- 05 Wide Scaling & Behavior Judge
- 06 Experiment & Result
- 07 한계점과 향후 연구 방향

Agent가 뭐야?

챗봇

사용자가 묻고 → 텍스트로 답함
"Passive"

에이전트

목표를 주면 → 스스로 판단하고 → 직접 행동
"Active"

수동적 응답형 모델에서 능동적 실행형 시스템으로의 패러다임 전환

Agent의 3대 구성요소

Agent의 눈

- Perception
- 스크린샷, DOM 트리, HTML 등을 통한 환경 이해

Agent의 뇌

- Memory & Planning
- 과거 행동 기록을 바탕으로 다음 스텝을 추론

Agent의 손

- Action
- API 호출, 마우스 클릭, 키보드 타이핑 등 환경에 직접 변화를 주는 인터페이스

용어 정리

Rollout

= 실행 경로
= 에이전트가 하나의 목표를 받아서 시작부터 끝까지 작업을 수행해 나간 독립적인 '한 번의 시도 과정'

Action Space

= 행동 공간
= 에이전트가 특정 환경(웹 브라우저, 터미널, 운영체제 등)에서 수행할 수 있는 모든 유효한 행동들의 집합

Observation Space

= 관측 공간
= 에이전트가 행동을 취한 직후, 환경으로부터 받아오는 피드백 정보의 범위

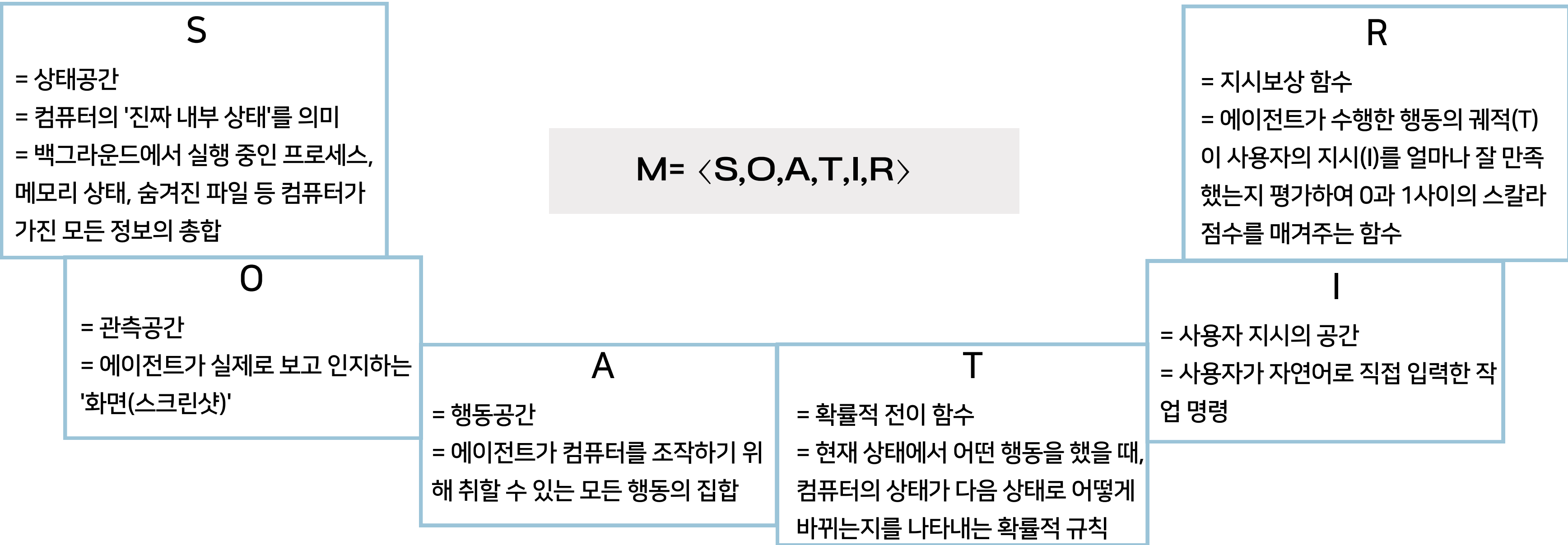
Trajectory

= 궤적
= 에이전트가 태스크를 수행하는 동안 거쳐간 모든 상태(State), 행동(Action), 그리고 관측(Observation)의 순차적인 기록(History)

기존 Agent의 문제점

POMDP(Markov Decision Process)

실제 컴퓨터 환경은 에이전트에게 매우 불리한 구조다. 이를 수학적으로 정의한 것이 바로 POMDP!



POMDP(Markov Decision Process)

어떤 구조길래?

$$M = \langle S, O, A, T, I, R \rangle$$

- 에이전트는 컴퓨터의 전체 상태(S)를 볼 수 없고 오직 모니터 화면(O)만 부분적으로 의존한다
- 행동(A)에 따른 결과(T)가 로딩 지연이나 팝업 등으로 인해 확률적으로 변한다

→ 오차 누적(Error Compounding)과 높은 분산(High Variance) 문제가 발생

1. 오차누적(Error Accumulation)

에이전트는 연속된 순차적 단계로 작업을 수행하기 때문에, 초기 단계에서 발생한 미세한 판단 오류나 잘못된 툴 콜(Tool Call)이 다음 단계의 입력으로 그대로 전달된다. 이로 인해 후속 작업 전체가 오염되며, 태스크가 진행될수록 원래 목표에서 멀어지는 눈사람 효과(Snowball Effect)가 발생해 수정이 불가능해진다.

2. 장기 호라이즌(Long-Horizon)

수십 단계 이상이 소요되는 복잡한 장기 프로젝트나 태스크에서, 에이전트는 거시적인 목표와 미시적인 세부 단계 사이의 일관성을 유지하기 어렵다. 전체 흐름 속에서 자신이 어디쯤 와 있는지, 어떤 가설을 검증해야 하는지 장기적인 메모리 관리가 부실해지면서 도중에 길을 잃거나 무한 루프에 빠지게 된다.

해결방법?

Scaling!

Scaling이 뭐야?

인공지능과 에이전트 분야에서 스케일링(Scaling)은 성능을 극대화하기 위해 모델 크기, 데이터 양, 혹은 연산량의 규모를 키우는 방식

Test-time Scaling

에이전트가 여러 개의 서로 다른 해결 경로 (Rollout)를 동시에 탐색하고 그중 가장 완성도 높은 궤적을 선택하게 만드는 방식

Pre-training Scaling

거대언어모델(LLM) 자체의 파라미터 크기나 학습에 사용된 데이터 양을 대폭 늘려 기초적인 문해력과 추론 능력을 전반적으로 끌어올리는 전통적인 방식

Compute Scaling

하나의 복잡한 태스크를 수행할 때 더 많은 연산 자원과 시간을 할당하여(예: 탐색 트리 확장, 내부 검증 루프 반복), 에이전트가 스스로 시행착오를 거치며 깊이 있게 고민하고 우회로를 찾도록 지원하는 방식

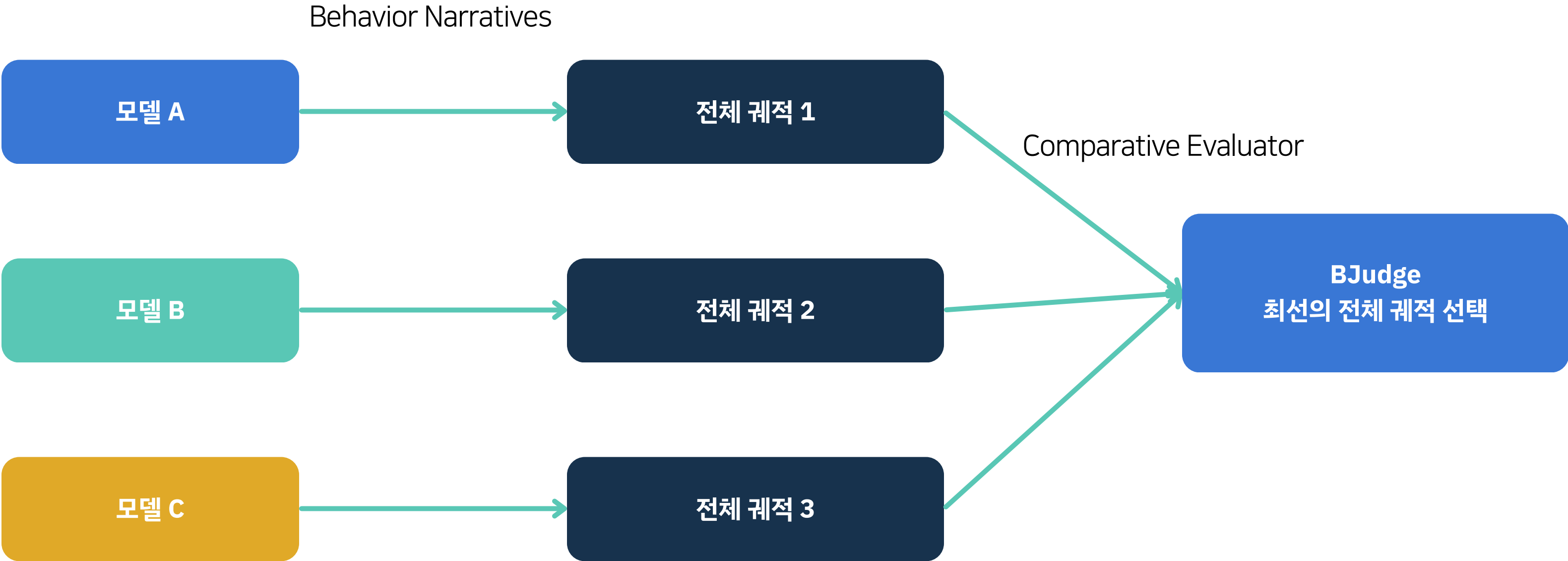
기존 연구들은 에이전트 하나를 믿고 딱 한 번 실행(Single-rollout)시키는 데 머물렀으나 불안정한 POMDP 환경 속에서 에이전트 하나가 처음부터 끝까지 실수 없이 완주하기란 너무나 어렵다

Test-time Scaling

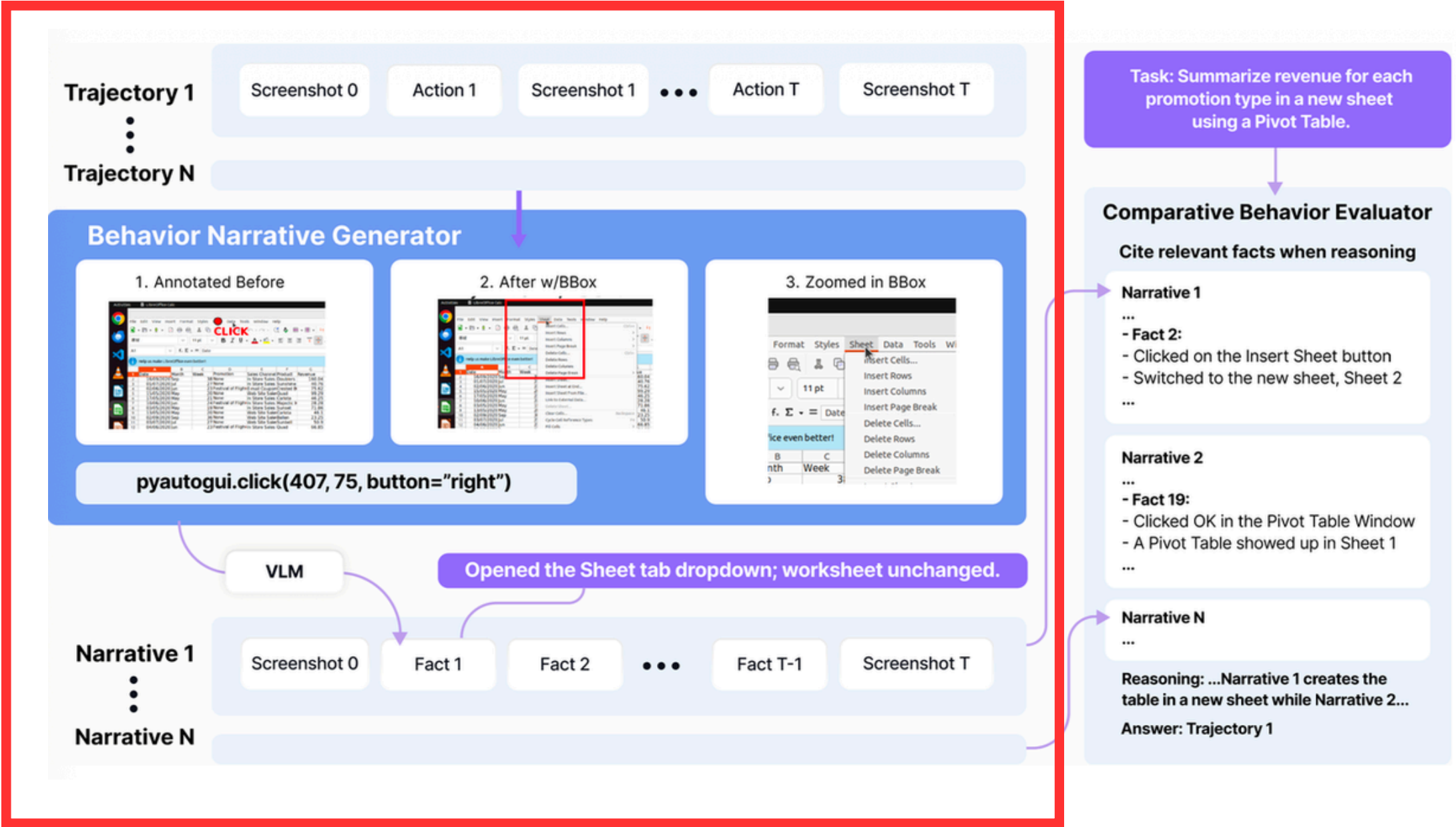
에이전트가 여러 개의 서로 다른 해결 경로 (Rollout)를 동시에 탐색하고 그중 가장 완성도 높은 궤적을 선택하게 만드는 방식

"하나만 믿고 가지 말고, 여러 개의 에이전트를 동시에 돌려서 여러 개의 실행 결과(Multiple rollouts)를 병렬로 확보하자!"

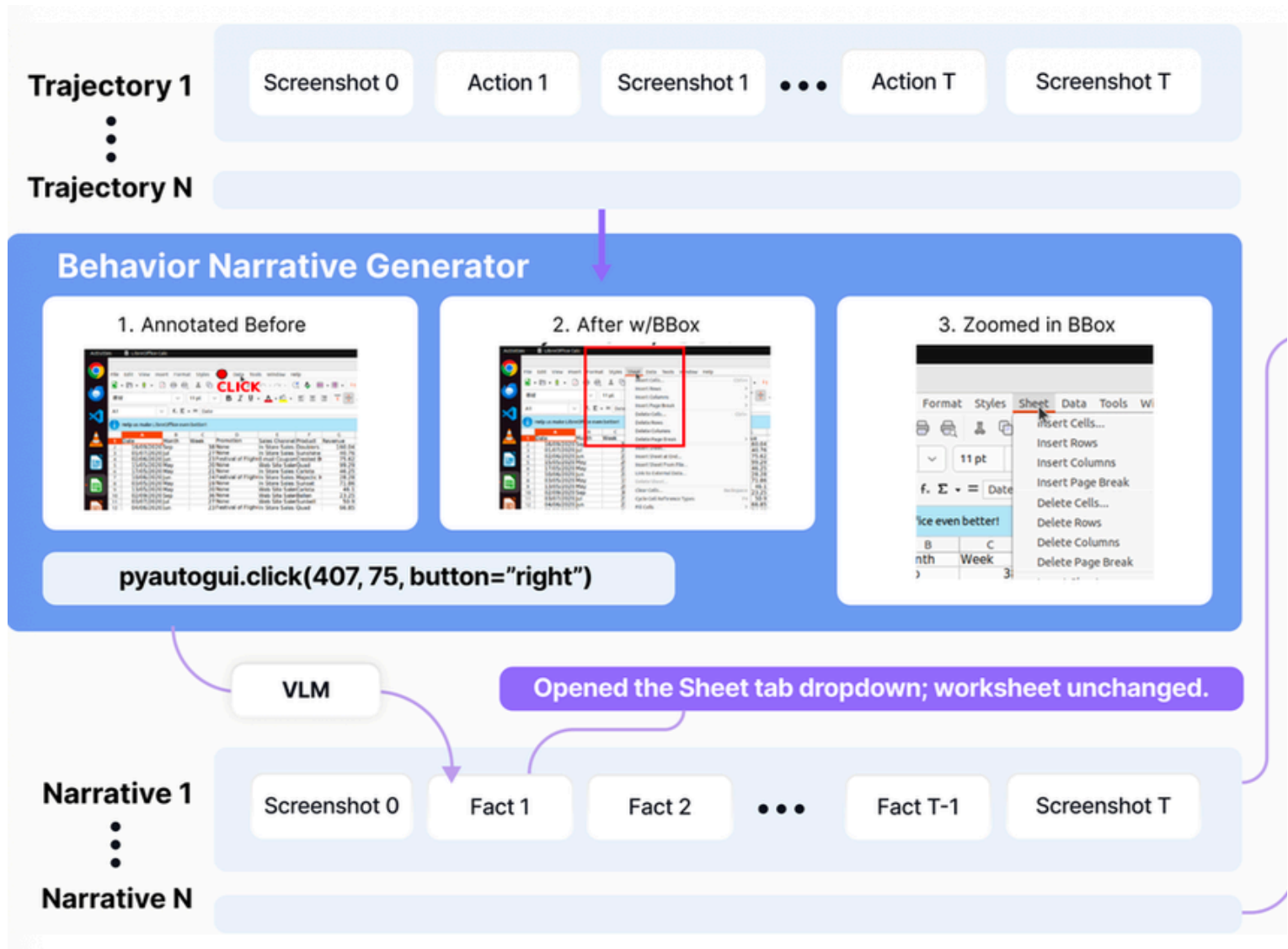
Wide Scaling & Behavior Judge



Behavior Narratives



Behavior Narratives



1. 입력 데이터: 각 에이전트의 실행 기록(Trajectory)에서 [이전 화면, 실행한 행동, 결과 화면]의 전이(Transition) 데이터를 입력받음.

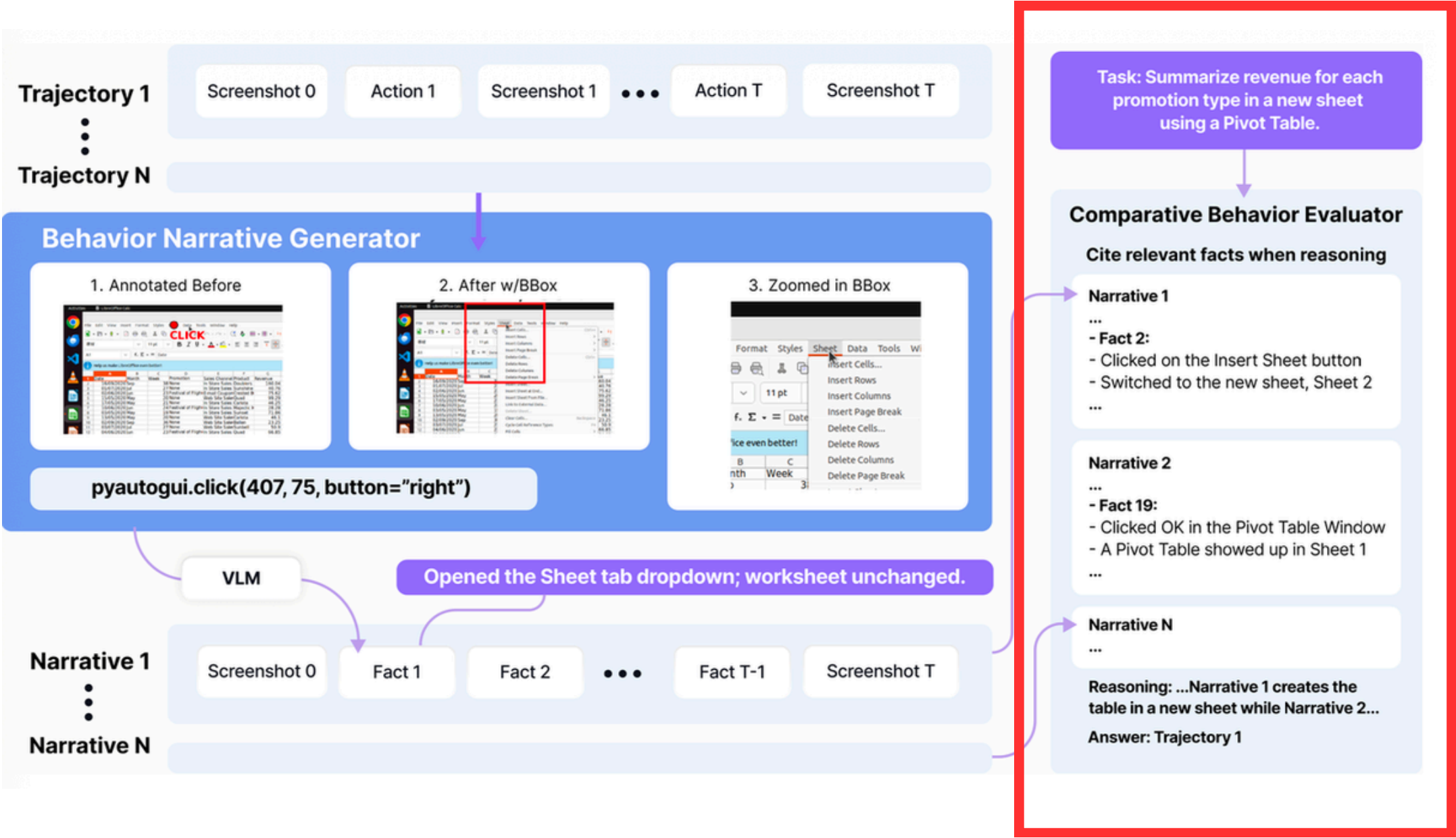
2. 시각적 증강 (Visual Augmentation):

- 행동 전 화면에 클릭 위치를 나타내는 빨간색 마커(CLICK)를 표시함.
- 행동 후 화면에서 포인터 주변 영역을 네모나게 지정하고(BBox), 해당 영역을 크게 확대(Zoom-in)하여 AI가 미세한 변화를 검증할 수 있게 함.

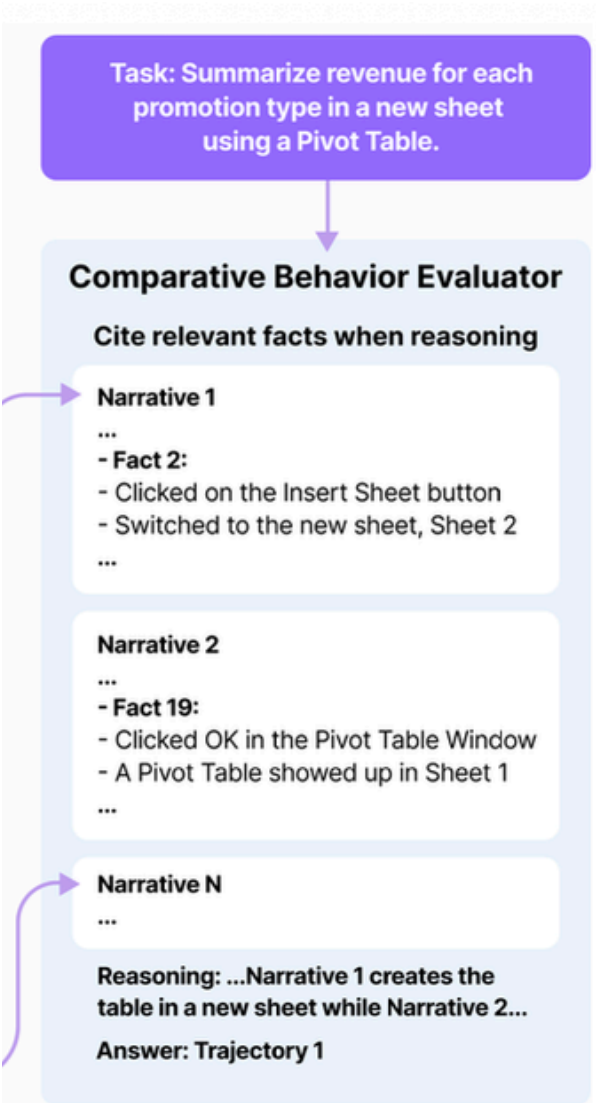
3. 핵심 사실(Fact) 도출: VLM을 통해 각 스텝의 변화를 분석하여 "시트 탭 드롭다운을 열었음"과 같은 간결한 사실 문장(Φ)으로 요약함.

4. 최종 서사 완성: 초기 화면(S_0)과 최종 화면(S_T), 그리고 단계별 사실들($\Phi_1, \Phi_2, \dots$)을 엮어 작업 관련 변경 사항만 담은 행동 서사를 생성함.

Comparative Evaluator(BJudge)



Comparative Evaluator(BJudge)



1. 입력 데이터: 사용자의 원래 작업 지시문(Task)과 생성된 여러 후보들의 행동 서사들(Narrative 1 ~ N)이 입력됨.
2. 추론 및 대조 (Reasoning): VLM 평가자가 개별 서사들을 따로 보는 것이 아니라, 한눈에 펼쳐놓고 각 서사의 핵심 사실(Fact 2, Fact 19 등)을 서로 인용하고 대조(Contrast)함.
3. 객관식(MCQ) 평가 방식: 단일 라운드 객관식 형태를 취하여, 토큰 효율성과 속도를 높이면서 후보 간의 차이를 명확하게 비교함.
4. 최종 정답 선택 (Answer): 지시사항을 가장 완벽하게 수행한 단 하나의 최우수 궤적(Trajectory 1)을 최종 정답으로 콕 집어 선택함.

Experimental & Results

1. Experimental Setup

- 주요 벤치마크: 우분투 환경의 OSWorld(361개 태스크)를 주력 무대로 삼고, OS 일반화 성능을 검증하기 위해 윈도우 기반 WindowsAgentArena(154개 태스크)와 안드로이드 기반 AndroidWorld(116개 태스크)를 함께 활용했습니다.
- 비교 모델(베이스라인): 기존 최고 성능의 오픈소스 프레임워크인 Agent S2, MobileUse, UI-Venus를 비롯해, 독립적 순위 평가 방식인 적응형 WebJudge 등을 비교군으로 설정했습니다.
- 모델 조합: 행동 서사의 사실(facts) 생성에는 Claude Opus 4.5를 사용하고, 최종 궤적 선택(BJudge)에는 GPT-5를 사용하여 각 파운데이션 모델의 강점을 극대화했습니다.

Experimental & Results

2. Agent S3 아키텍처

기존 Agent S2의 병목을 해결하기 위해 도입된 Agent S3는 두 가지 핵심 설계 변경을 거쳤다.

- 코딩 에이전트 네이티브 통합:

대규모 데이터 처리나 파일 변환 등 픽셀 정밀도가 떨어지는 무거운 작업은 마우스 GUI 조작 대신 Python/Bash 코드 실행(샌드박스 환경)으로 처리한다. 오케스트레이터 없이 GUI 에이전트의 액션 공간에 직접 내장되어 스스로 코딩 호출 시점을 판단한다.

- 계층 구조 제거 (Flat Policy):

기존의 복잡한 '매니저-작업자' 계층 계획을 폐기하고 단일 평면 정책을 적용했다. 현대 VLM의 뛰어난 문맥 유지력 덕분에 실시간 재계획이 가능해져, 작업 완료 시간이 62.4% 단축되고 LLM 호출 횟수가 52.3% 감소하는 폭발적인 효율성 향상을 기록했다.

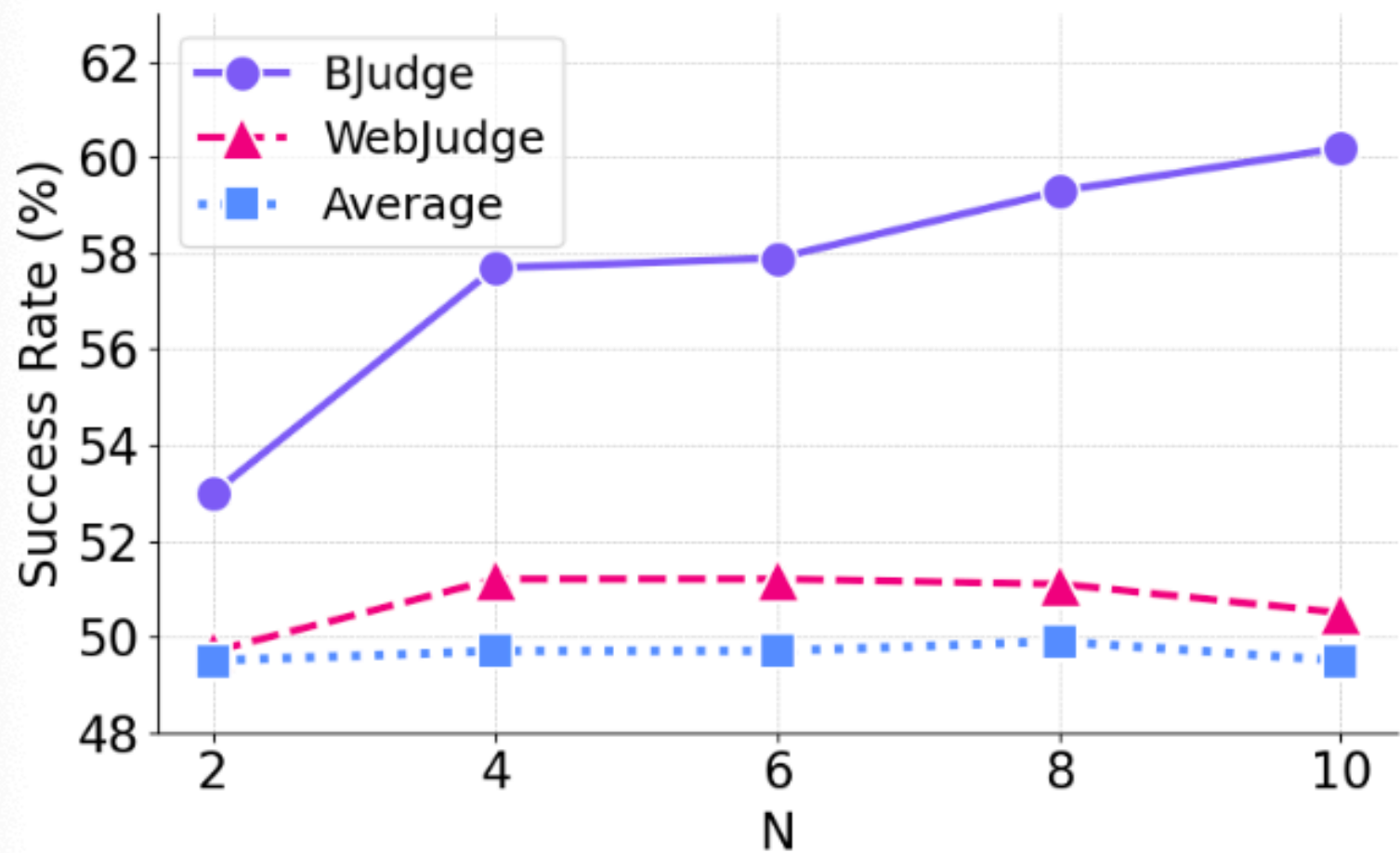
Experimental & Results

3.Result

- OSWorld 벤치마크 100스텝 기준 최고 성공률 72.6%를 달성하여, 기존 인간 평균 수행 능력인 72.36%를 공식적으로 돌파했다

Method	Model	100-step
Agent S2 (Agashe et al., 2025b)	GPT-5	48.8
Jedi-7B (Xie et al., 2025)	o3	51.0
CoAct-1 (Song et al., 2025)	OAI CUA + o3 + o4-mini	59.9
Ours		
Agent S3	o3	61.1
Agent S3	GPT-5 Mini	49.8
Agent S3	GPT-5	62.6
Scaling Results		
GTA1 (step-wise scaling) (Yang et al., 2025b)	o3	53.1
GTA1 (step-wise scaling) (Yang et al., 2025b)	GPT-5	63.4
Agent S3 w/ WebJudge (N=10) (Xue et al., 2025)	GPT-5 Mini	50.4
Ours		
Agent S3 w/ BJudge (N=10)	GPT-5 Mini	60.2
Agent S3 w/ BJudge (N=10)	GPT-5	69.9
Agent S3 w/ BJudge (N=10)	GPT-5 + Opus 4.5	72.6

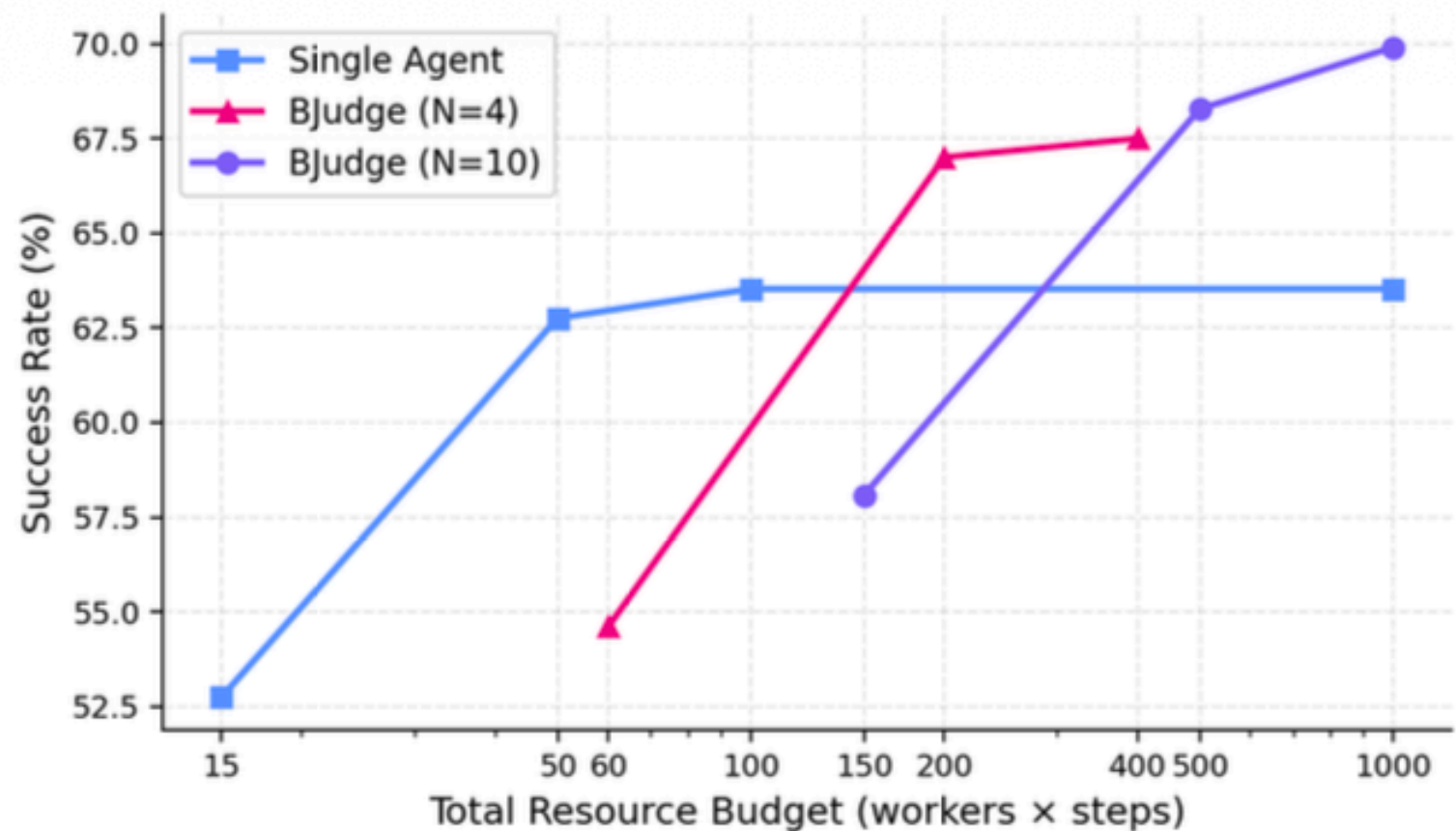
Experimental & Results



동일 롤아웃 예산 하에서의 성능 비교

- 그래프 내용: 후보 롤아웃 개수(N)를 동일하게 2개, 4개, 6개, 8개, 10개로 늘려갈 때, 기존 점수 매기기 방식(WebJudge)과 본 논문의 비교 평가 방식(BJudge)의 성공률 변화를 대조함.
- 핵심 시사점:
 - 단순 개별 점수 방식(WebJudge)은 후보를 늘려도 성능이 금방 정체되거나 오히려 하락함.
 - 반면 BJudge는 후보 개수가 늘어날수록 성공률이 꾸준히 우상향(Wide Scaling)하여, 궤적들을 서로 대조하여 고르는 방식이 훨씬 우수함을 입증함.

Experimental & Results



총 자원 예산에 따른 성능 분석

- 그래프 내용: 모델 호출 횟수나 토큰 비용 등 전체 연산 자원 예산이 제한된 상황에서 BJudge가 성능을 얼마나 효율적으로 뽑아내는지 보여줌.
- 핵심 시사점:
 - 무조건 덩치가 큰 모델 하나로 오래 끄는 것보다, 적절한 수준의 후보군을 생성한 뒤 정밀한 서사 기반 비교 평가를 거치는 것이 한정된 자원(Cost-Accuracy Trade-off) 대비 가장 높은 성공률을 달성한다는 점을 보여줌.

한계점과 향후 연구 방향

- 비용 및 지연 시간 (Cost and Latency Trade-off):
 - 여러 개의 롤아웃을 병렬로 생성하고 VLM 판정자(BJudge)를 구동해야 하므로, 단일 에이전트 방식에 비해 더 많은 컴퓨팅 자원, API 토큰 비용, 그리고 총 실행 시간(latency)이 소모된다.
- 기반 에이전트 성능 의존성 (Base Policy Dependency):
 - BJudge는 여러 후보 궤적들 중에서 정답을 '선택'하는 구조이므로, 애초에 후보군을 생성하는 기반 에이전트(Agent S3 등)가 올바른 답을 후보 안에 하나도 포함시키지 못했다면 심사할 수 있는 여지가 없어 성능의 상한선이 제한된다.
- 웹 환경 및 모바일 제약 (Environment Restrictions):
 - 샌드박스 환경의 보안 정책이나 로그인/인증(예: Google Drive 등 자격 증명)이 필요한 복잡한 시나리오, 혹은 프로그램 실행 도구가 제한된 모바일(AndroidWorld) 환경 등에서는 코딩 에이전트 활용이 제한되어 성능 극대화에 걸림돌이 존재한다.

감사합니다

Question?